Python for computational Problem Solving · Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming

DRAFT — UE26CS151A · Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming · Python Classes and Objects

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Python Classes and Objects
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Python Classes and Objects

1 Python Classes and Objects

Study focus: Python Classes and Objects. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000022 · EV0000023

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Python for computational problem solving.

Key details

- Today we will try to learn how do we create a class.
- How do we work with the class?
- In order to create a class, I need to make use of a keyword called as class.
- I need to make use of a keyword called as a class.
- Followed by you will also have to specify what is the name of your particular class.
- You can give your name itself as a name of the class, but.
- it is always better to name it in a proper convention.
- The reason is whenever you are coming back in order to look at for look at your program and again, it should make sense in order to tell that what exactly you were trying to do.
- I will be telling a step by step at this point.
- Try to understand that I do not want to do anything.
- That's why I say it as a pass.
- Next, what I will be doing, nothing else, I'll just say a print.
- I'll tell what is the type of my class.
- Type of my class.
- The reason is we have said the class that you are going to be creating is a type.
- Because type specifies what you have to do, but it never specifies how you have to do.
- a small correction in the previous video I looked in the spelling of specifies was written wrong.
- Please correct it.
- What is I have said it as class itself.

- And find out.
- EX1's type is type itself.
- I need to create my own type.
- I'm trying to create my own type.
- So this is our first program.
- How do we create a class?
- We will extend this in order to make our life simple, let me comment it like this.
- Okay, so now the next point that we have to understand is a class has a behavior.
- Class has a behavior.
- And I will create a function.
- I do not want to do anything other than just writing a print statement.
- This is fine.
- Nothing else that I want to do.
- I can access it by making use of a function name.
- Dot the function name.
- Class name dot the function name.
- We said that a class has a behavior.
- Behavior in the sense it's a function.
- And if I say what is the behavior of a student, he has to listen to the class.
- What is the behavior of a teacher?
- That's the functionality.

Example or application from the lecture

- The name of my class is example one.
- We'll look with an example.
- So I will say class EX2 is what I will say because we are into the second example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000022 · EV0000023 · EV0000024 · +1 source units in pack.json

Core explanation · 05:00–09:59

A class has a behavior and the behavior that I defined as what?

Key details

- The function f1.
- It has to be done using your ex2 dot f1.
- This I call it as invocation of a function.
- A function is getting invoked.
- So we'll go ahead with the next one.
- The next one is a class also has.
- Class also has attributes.
- Attributes in the sense fields.
- It can also be a field or it can be a variable.
- Clause, let me do it as EX3, because this is my EX3.

- Okay because this is my variable.
- Class concept something.
- Hope this is fine for you.
- I do not want to do anything else in this, so let it be as simple as possible.
- I have to access A, but it is a part of which class?
- It's a part of EX3 class.
- It says that class concept.
- maybe I did not execute the output for the previous one.
- And one thing that should always needs to be remembered is the indentation.
- It's an header.
- Important thing is the indentation.
- This is called as an header.
- The suit will follow the required indentation.
- I always use indentation, which is going to be the tab itself.
- This statement says it is out of the class.
- Anything within the class with has with some indentation.
- I say that this is a body of the class or the suit.
- S-U-I-T, we usually pronounce it as what?
- Suite of the class.
- And this is called as an header.
- Anything that ends with the colon, hope you remember that it is called as what?
- This itself becomes an header, and this becomes a body of this particular header, and so on.
- What we have done, we created a class, we created a behavior for the class, we also created an attribute for a particular class.
- We create a variable for a class which is of type of that particular class.
- What does that mean?
- See, when you when you I when I say you are a PAS student.
- You are a PES student.
- You belong to A section.
- I'll just assume it as it is sixteen students.
- How I will name each of the students.
- At this point I may say role number one, role number two, role number three, four, and so on.

Example or application from the lecture

- We will see the output that I'm expecting is what in F1 because when on this example is executed and here I need to get the output called as class concept.
- Let's move on to the next one.
- And suppose you belong to A section.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000026 · EV0000027 · EV0000028 · +1 source units in pack.json

Core explanation · 10:00–14:59

I can say that section A is a class which has so many number of variables, and these variables are of type A section.

Key details

- A section type.
- This is also called as an object.
- It is also called as an instance.
- We are going to be creating an instance.
- Do you want to do anything?
- No, I don't want to do anything at this point.
- as we are talking about A section.
- Syntax that you have to follow whenever you're trying to create an object.
- Print type of A.
- Type of A is what?
- It A belongs to your EX4 class and it is a part of your main class.
- The moment you create a class.
- Hope all the concepts are clear to you till now.
- A class, it doesn't have any implementation.
- I just created an object.
- Internally, there is a special function that gets created.
- And this special function is called as underscore, underscore, init underscore, underscore.
- You should always remember this, even though you do not need to create at this point internally, what has happened?
- Internally there is an underscore, underscore in it, underscore, underscore.
- It gets created.
- in conceptually or internally what exactly this means.
- This means that there is a function, special function gets created which has an argument A.
- We will complete this particular program.
- We will explicitly create underscore in it underscore.
- Okay, once an object is created, internally there is a method that gets executed, which is called as what?
- A special function, and we will give a name to the special function, and the name of the special function is called as constructor.
- It does a constructive job.
- Let me remove the comment and put it here.
- Let me create a class.

Example or application from the lecture

- So I will also take up an example called as A is equal to I need to make use of an EX4, which is the name of the class itself.
- Let's find out what is a type of A.
- We will try to understand with an example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000030 · EV0000031 · EV0000032 · +1 source units in pack.json

Core explanation · 15:00–19:59

Okay, because I said a special function that gets created.

Key details

- Oh what do we print?
- We usually say it as constructor.
- Instead of typing such a big word, I just said it as construct.
- What do I do?
- I'll just create a object.
- Of type what?
- Nothing else that I really want to do here.
- And if you can see it throws me an error.
- It is throwing me an error.
- The reason is internally it looks like this, right?
- It looks like underscore, underscore of A.
- Okay, so an instance is passed as the first argument to this particular constructor.
- Let me repeat, when you do this, the instance itself is passed as a first argument.
- I need to pass something.
- Let me repeat once again, the moment I create an object, internally the constructors gets executed.
- Fine, even though this constructor has an imp implicit in nature, but what will happen is the object or an instance becomes.
- As an argument.
- It's an argument.
- There is nobody here to hold my argument.
- Because of which I have to explicitly specify a particular parameter.
- I can specify a parameter.
- There is no problem with this.
- It's absolutely working fine.
- You can give any name to the parameter, but by practice we always use something called as self.
- This is normally it we use it.
- Okay there is no role because we previously we executed with the parameter itself.
- There was no harm in executing, but why self?
- Self is generally used.
- It's a.
- It's by practice we can say.
- You use your own name, it is absolutely fine.
- And we will try to identify what we are trying to say.
- Okay because I said internally it is passed as what?
- A, which means self is A.

- A is self.
- The output that I'm expecting is both of them should print and give me the same answer.
- Okay, and if you can see both of them are pointing to the same ones, which means A is same as self, self is same as A.
- I hope there are no doubts in this case.
- Sorry if there are any spelling mistakes.
- It's not intentional.
- So define why do I need to do init method?
- It's a constructor.
- So what should go to my init method?
- The first parameter should be what?
- To hold the instance of this particular class.
- As I'm talking about rectangle, I also speak about what?
- I also speak about breadth.

Example or application from the lecture

- Let's execute.
- Let's print self here.
- This probably this is the last example that I would be taking in order to explain the concept of creation of class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000034 · EV0000035 · EV0000036 · +1 source units in pack.json

Core explanation · 20:00–24:59

Okay, so what is the next thing that I have to do?

Key details

- Length of which one?
- What length of which one is what I'm asking?
- Of that particular instance.
- How do I point that instance?
- Self dot length okay this is equal to L.
- Fine, hop this is clear.
- I hope there are no problems here.
- I will say print lent.
- Let me do this.
- There is nothing because what happens when I execute this?
- Header is process.
- I never get into the body until we call this particular an object for this particular class gets created.
- I'm going to write the program completely.
- I what do we do?
- We just print a length and a breadth.

- Nothing much we want to do it here.
- Length into breadth is what you can say, but that we will try to do it later on.
- I'll say self.length.
- Comma self dot breadth.
- The reason why I commented also I will let at this point, please just bear with me.
- R is the object for this rectangle.
- How do I need to create an object?
- Use the name of the class.
- C T A N G L E, a big name.
- Internally, this is going to be like underscore in it underscore of R.
- But what do I need?
- I need to.
- find what is the length and what is the breadth.
- I want to print both of them.
- I will pass the values like 10 comma twenty.
- 10 comma 20.
- Okay, so ten and twenty because we are not doing anything, we're just trying to print this.
- Push it here.
- And if you can see name length is not defined as what it was trying to say.
- But when I commented this, I'm getting the output properly.
- The main reason is.
- Whenever you're making use of a variable, you should always use a fully qualified name.
- Length of which one?
- Length of this particular object.
- It should be self.length.
- Let me execute both of them.
- Length is 10, length is 10, comma 20.
- In this class, a pretty good way of we have understood so many things.
- We created an empty class, we came to know that class has a behavior, class has an attributes, okay.
- In this case, we came to know that there is an.
- Invalid function that gets invoked on creation of a particular object, which is called as a constructor.
- We have used constructor in order to initialize the variables.
- So in the next class, what we will do is we will try to find out how I can add.

Example or application from the lecture

- Let's quickly just execute this to find out what is going to happen.
- So now this is an example where a particular class has a function, but this is a built-in function.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000038 · EV0000039 · EV0000040 · +1 source units in pack.json

Core explanation · 25:00–29:59

function to this particular class.

Key details

- Because we have done the function separately and we have done the attributes separately, but we will try to find out how do we add a function to this particular class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000042 · EV0000043 · EV0000044 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Python Classes and Objects transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Python Classes and Objects transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
3. 10 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
4. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
5. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
6. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
7. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00010	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00011	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00012	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00013	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

C Revision and retrieval practice